3.3_apply

`*apply` family

Imagine you have a list of vectors and you want to apply the same function(s) to each element of the list:

- I apply the function manually, selecting the elements
- a `for` loop that iterates over the elements of the list and applies the function(s)

...

```
my_list <- list(
    var1=rnorm(n = 100, mean = 0, sd = 1),
    var2=runif(n = 100, min = 0, max = 1),
    var3=rnorm(n = 100, mean = 0, sd = 1),
    var4=rnorm(n = 100, mean = 0, sd = 1)
)
```

*apply family

Let's compute mean, median and std

```
# initialize the vectors with arb
# but as many as we need
means <- 1:length(my_list)
medians <- 1:length(my_list)
stds <- 1:length(my_list)

# Loop
for(i in 1:length(my_list)){

  means[i] <- mean(my_list[[i]])
  medians[i] <- median(my_list[[i
  stds[i] <- sd(my_list[[i]])
}
```

Result

```
means
```

```
[1]  0.05671296
0.47225496 -0.22413846
0.07344355
```

```
medians
```

```
[1] -0.08141012
0.45878207 -0.14936708
0.10168012
```

```
stds
```

```
[1] 0.9991443 0.2837145
1.0871693 0.9811374
```

`*apply` family

Everything works! but:

- writing the for loop is laborious, with indices both for the list and for the vector we are filling
- we need to pre-allocate variables (for speed reasons)
- 8 lines of code (for this simple example)

*apply family

R has a family of apply functions such as **lapply**, **sapply**, etc. that let us get the same result in a more concise, fast and simple way:

```
means <- sapply(my_list, mean)
medians <- sapply(my_list, median)
stds <- sapply(my_list, sd)

means
```

```
      var1       var2       var3       var4 
0.05671296 0.47225496 -0.22413846 0.07344355 
```

```
medians
```

```
      var1       var2       var3       var4 
-0.08141012 0.45878207 -0.14936708 0.10168012 
```

```
stds
```

`*apply` family

apply(< list > , < function >)

- What can the **list** be?
  - a list
  - a dataframe
  - a vector
- What can the **function** be?
  - base or imported from a package
  - custom
  - anonymous

`*apply` family

Before analysing the `*apply` family, I think a further parallel with the for loop we have seen is useful. `apply` is nothing more than a slightly simplified `for` loop.

`for` loop

```
1 vec <- 1:5
2 for(i in vec){
3   print(i)}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

`sapply`

```
1 vec <- 1:5
2 res <- sapply(vec, print)
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

*apply family - custom function

We can also use functions we created ourselves:

```r
center_var <- function(x){
  x - mean(x)
}

my_list <- list(
    vec1 = runif(n = 10, min  = 0, max = 1),
    vec2 = runif(n = 10, min  = 0, max = 1),
    vec3 = runif(n = 10, min  = 0, max = 1)
)

lapply(my_list, center_var)
```

```
$vec1
 [1]  0.35622590  0.40579508 -0.13355142 -0.26581224
0.11323090 -0.31519911
 [7] -0.31005045 -0.07008290  0.25763143 -0.03818719
```

```
$vec2
 [1]  0.28614303  0.34409264  0.38747787 -0.53213280
0.14114328 -0.17051661
 [7] -0.39408174  0.07340621  0.27975540 -0.41528730

$vec3
```

`*apply` family - implicit vs. explicit

So, just as the `for` loop written as **i** in **vec** assigns to **i** one **element** of the **vec** object at a time, internally the `*apply` functions take the **first element** of the **input** object (list) and directly **apply** the function we have chosen.

implicit `sapply`

```
1 vec <- 1:5
2 res <- sapply(vec, p
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

explicit `sapply`

```
1 vec <- 1:5
2 res <- sapply(vec, function(i) pr
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

*apply family - anonymous function

An anonymous function is a function that is not saved in an object but written to be **executed directly**, inside other functions that allow it:

```
1 lapply(my_list, function(x) x - mean(x))
```

```
$vec1
 [1]  0.35622590  0.40579508 -0.13355142 -0.26581224
0.11323090 -0.31519911
 [7] -0.31005045 -0.07008290  0.25763143 -0.03818719

$vec2
 [1]  0.28614303  0.34409264  0.38747787 -0.53213280
0.14114328 -0.17051661
 [7] -0.39408174  0.07340621  0.27975540 -0.41528730

$vec3
```

x is just a **placeholder** (analogous to i) and can be any letter or name!

All the types of `*apply`

- `lapply()`: the basic function
- `sapply()`: simplified-apply
- `tapply()`: rarely used, useful with factors
- `apply()`: useful for dataframes/matrices
- `mapply()`: multivariate version, uses several lists at the same time
- `vapply()`: used inside functions and packages

lapply

lapply stands for list-apply and always returns a list, applying the function to each element of the input list:

```
1  res <- lapply(my_list, mean)
2  res
```

```
$vec1
[1] 0.5735488

$vec2
[1] 0.5850786

$vec3
[1] 0.514791
```

```
1  class(res)
```

```
[1] "list"
```

sapply

sapply stands for simplified-apply and (tries to) return a simpler version of a list, applying the function to each element of the input list:

```
1 res <- sapply(my_list, mean)
2 res
```

```
     vec1      vec2      vec3
0.5735488 0.5850786 0.5147910
```

```
1 class(res)
```

```
[1] "numeric"
```

apply

apply works specifically on dataframes or matrices, applying a function to the rows or the columns:

```r
my_df <- data.frame(x1 = runif(n = 5,min = 1,max = 10),
                    x2 = runif(n = 5,min = 3,max = 4))
my_df
```

```
        x1       x2
1 4.902174 3.639349
2 1.068568 3.768333
3 3.977162 3.767391
4 5.668196 3.190103
5 3.749397 3.377448
```

apply

Apply the mean function to all rows (1):

```
1 apply(my_df, MARGIN = 1, FUN = mean)
```

```
[1] 4.270762 2.418450 3.872276 4.429150 3.563423
```

Apply the mean function to all columns (2):

```
1 apply(my_df, MARGIN = 2, FUN = mean)
```

```
      x1       x2 
3.873099 3.548525 
```

tapply

tapply lets us apply a function to a vector, splitting the vector according to a categorical variable:

```
vec <- rnorm(75)

index <- rep(c("a", "b", "c"), each = 25)

tapply(vec, INDEX = index, mean)
```

```
          a           b           c
-0.06327193 -0.03021544 -0.17966927
```

Here *INDEX* is a character vector or a factor.

tapply

In this case we compute the mean for each age category:

```
1 my_df <- readr::read_csv("data/mydf_2.csv")
2 head(my_df)
```

```
# A tibble: 6 × 4
     id   age age_cat       age_z
  <dbl> <dbl> <chr>         <dbl>
1     1    29 young       -0.188
2     2    18 adolescent  -1.17
3     3    27 young       -0.366
4     4    40 adult        0.795
5     5    30 young       -0.0983
6     6    38 adult        0.616
```

```
1 tapply(my_df$age, my_df$age_cat, mean)
```

```
adolescent      adult      young
  16.50000   40.86667   24.55556
```

vapply

vapply is a more robust version of the previous ones from a programming point of view. In practice it lets (and requires) us to specify in advance the type of data we expect as a result:

```
1 vapply(my_list, FUN = mean, FUN.VALUE = numeric(length = 1
```

```
     vec1      vec2      vec3
0.5735488 0.5850786 0.5147910
```

FUN.VALUE = numeric(length = 1): indicates that each result is a single numeric value.

mapply

mapply lets us handle **several lists at the same time** for more complex scenarios. For example, we want to use the **rnorm()** function to generate 4 samples combining different means and standard deviations:

```r
mus <- list(10, 20, 30, 40)
stds <- list(1, 2, 3, 4)

mapply(function(x,y) rnorm(n = 5, mean = x, sd = y),
       mus, stds, SIMPLIFY = FALSE)
```

```
[[1]]
[1] 10.500950 11.995598  8.955502  9.376923  8.577450

[[2]]
[1] 20.56899 18.50954 19.02368 22.09763 18.91090
```

```
[[3]]
[1] 28.21941 32.59818 30.05078 31.55771 28.69693

[[4]]
[1] 33.79673 40.84147 44.35516 40.30744 41.93724
```

IMPORTANT: all the lists included must have the same length!

mapply

mapply(function(x,y) rnorm(n = 5, mean = x, sd = y), mus, stds, SIMPLIFY = FALSE)

- `function(...)`: an anonymous function, as we saw before, which can have n elements
- `rnorm(n = 5, mean = x, sd = y)`: the actual anonymous function, where we have the placeholders x and y
- `mus, stds`: the lists corresponding, **in order**, to the placeholders, so x = mus and y = stds
- `SIMPLIFY = FALSE`: simply says to return a list and not to try (like sapply) to simplify the result

`mapply` as a `for`

The same result (in a more verbose way) can be obtained with a `for` loop, using the iterator i several times:

```
mus <- list(10, 20, 30)
stds <- list(1,2,3)
res <- vector(mode = "list", length = length(mus)) # empty

for(i in 1:length(mus)){
  res[[i]] <- rnorm(n = 6, mean = mus[[i]], sd = stds[[i]]
}
res
```

```
[[1]]
[1] 10.080162  9.749474 10.435478  9.733762  9.321152
10.788030

[[2]]
[1] 20.76959 19.95664 22.36724 16.88161 22.37280 20.24247
```

```
[[3]]
[1] 30.22561 27.78725 29.21119 24.91813 27.31852 28.30777
```

`replicate`

This function lets us repeat an operation n times, without using an iterator or a placeholder.

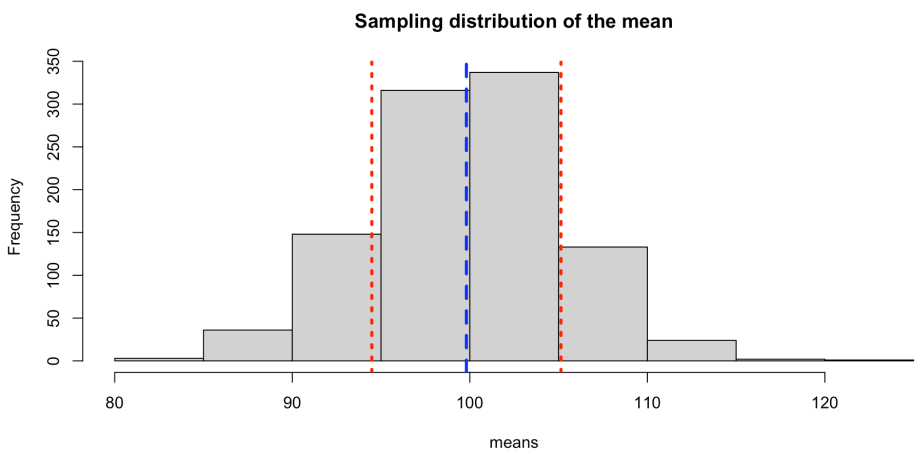
```
replicate(n, expr)
```

- ***`n`*** is the number of repetitions
- ***`expr`*** is the portion of code to repeat

replicate

We sample 1000 times from a normal distribution and compute the mean, AKA the sampling distribution of the mean

```
 1 nrep <- 1000
 2
 3 nsample <- 30
 4
 5 mu <- 100
 6
 7 sd <- 30
 8
 9 means <-
10   replicate(
11     n = nrep,
12     expr = {mean(
13       rnorm(nsample, mu, sd
14     )
```



Now let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR